Activity A: Predict and explain

Statement	Exception	Checked/Unchecked	Prevention
Integer.parseInt("ABC");	NumberFormatException	Unchecked	Validate the text before parsing, or wrap the call in try/catch and report the bad value.
String name = null; name.length();	NullPointerException	Unchecked	Check if (name != null) before calling a method on it.
new java.io.FileReader("missing.txt");	FileNotFoundException	Checked	Catch it in try/catch (or declare throws), or check that the file exists with File.exists() first.

OutOfMemoryError: It is an Error, not an Exception, and it means the JVM has run out of memory, so the program may be left unstable. Catching it just to carry on does not free any memory and can hide the real problem; the better fix is to reduce memory use or increase the heap.

ACTIVITY B:

MARKIMPORTER.JAVA

```
static void importMarks(String filename) throws IOException {
    Set<String> seen = new HashSet<>();
    try (BufferedReader br = new BufferedReader(new FileReader(filename))) {
        String line;
        int lineNo = 0;
        while ((line = br.readLine()) != null) {
            lineNo++;
            try {
                String[] parts = line.split(",");
                if (parts.length != 2) {                    // TODO 1
                    throw new IllegalArgumentException("Expected 2 fields but found " + parts.length);
```

```
            }
            String id = parts[0].trim();
            int mark = Integer.parseInt(parts[1].trim());  // TODO 2
            if (mark < 0 || mark > 100) {
                throw new IllegalArgumentException("Mark out of range 0..100: " + mark);
            }
            if (!seen.add(id)) {                          // TODO 3
                throw new DuplicateStudentException("Duplicate student ID: " + id);
            }
            System.out.println("Accepted: " + id + " -> " + mark);  // TODO 4
        } catch (NumberFormatException e) {
            System.out.println("Line " + lineNo + " skipped: mark is not a number.");
            logger.log(Level.WARNING, "NumberFormatException on line " + lineNo + ": " + e.getMessage());
        } catch (DuplicateStudentException e) {
            System.out.println("Line " + lineNo + " skipped: " + e.getMessage());
            logger.log(Level.WARNING, "DuplicateStudentException on line " + lineNo + ": " + e.getMessage());
        } catch (IllegalArgumentException e) {
            System.out.println("Line " + lineNo + " skipped: " + e.getMessage());
            logger.log(Level.WARNING, "IllegalArgumentException on line " + lineNo + ": " + e.getMessage());
        }
      }
    }
}
```

2.THE CUSTOMER EXCEPTION:

```
class DuplicateStudentException extends RuntimeException {
    public DuplicateStudentException(String message) {
        super(message);
    }
}
```

3.ON SKIP OR STOP:

I chose to skip invalid records and continue. A single bad line, such as a non-numeric mark, an out-of-range mark or a duplicate ID, should not stop valid marks for other students from being imported. Each skipped record is reported to the user in plain language and logged with its exception type and cause, so nothing is lost silently. Problems that affect the whole file, such as a missing or unreadable file, still stop the import, because there is nothing to continue with.

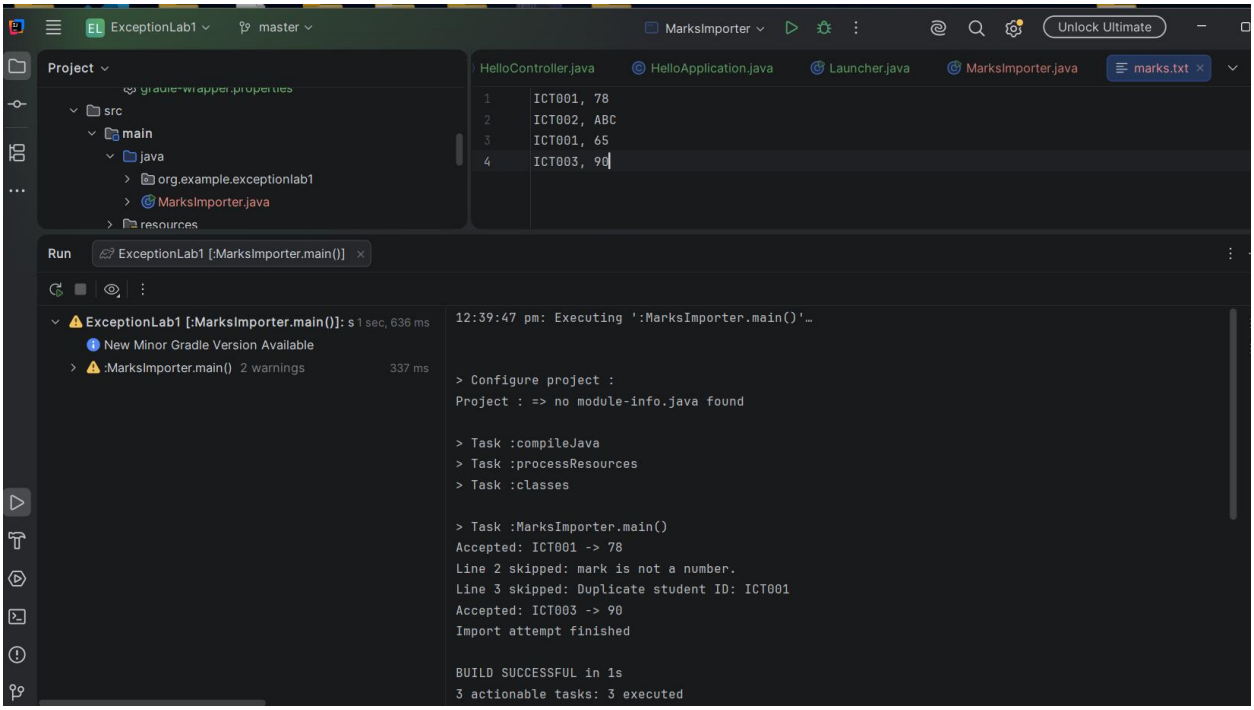
ACTIVITY C:

1. THE MAIN METHOD:

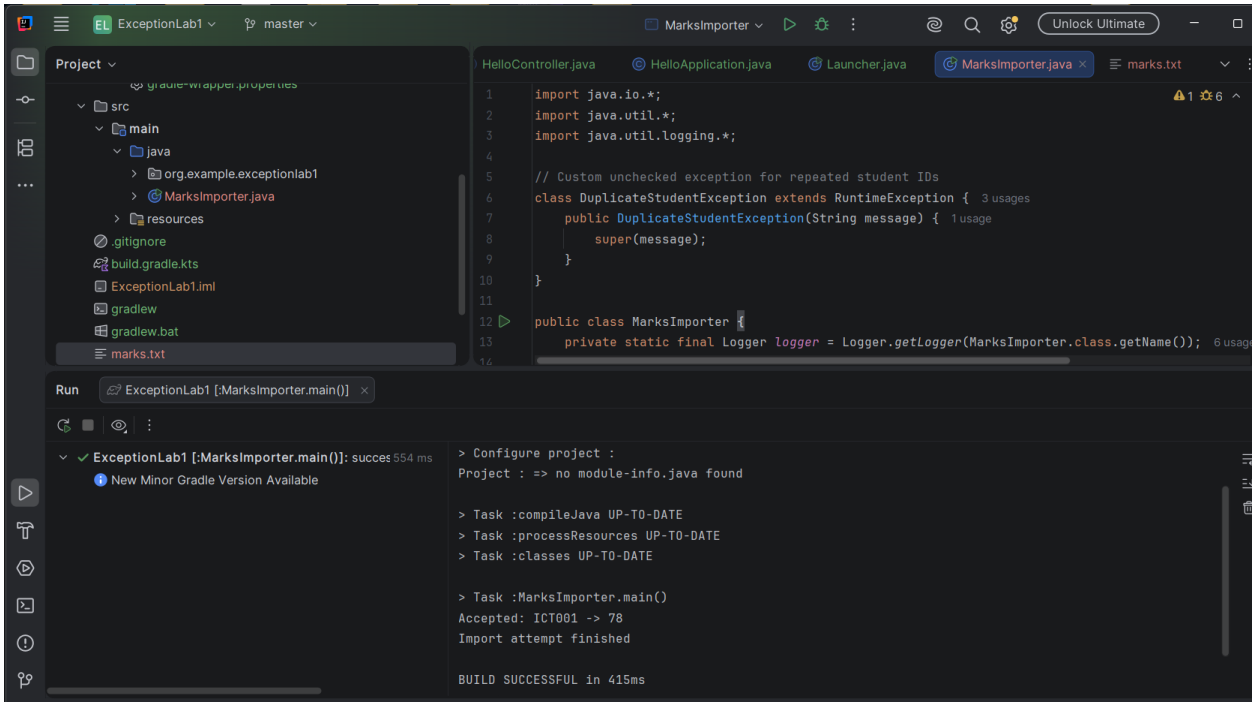
```
public static void main(String[] args) {
    try {
        importMarks("marks.txt");
    } catch (FileNotFoundException e) {   // specific before general
        System.out.println("Marks file not found; check the file name.");
        logger.log(Level.SEVERE, "FileNotFoundException: " + e.getMessage());
    } catch (IOException e) {
```

```
            System.out.println("Could not read the marks file. Please try again.");
            logger.log(Level.SEVERE, "IOException: " + e.getMessage());
        } catch (IllegalArgumentException | DuplicateStudentException e) {
            System.out.println("Invalid data stopped the import.");
            logger.log(Level.SEVERE, e.getClass().getSimpleName() + ": " + e.getMessage());
        } finally {
            System.out.println("Import attempt finished");
        }
}
```

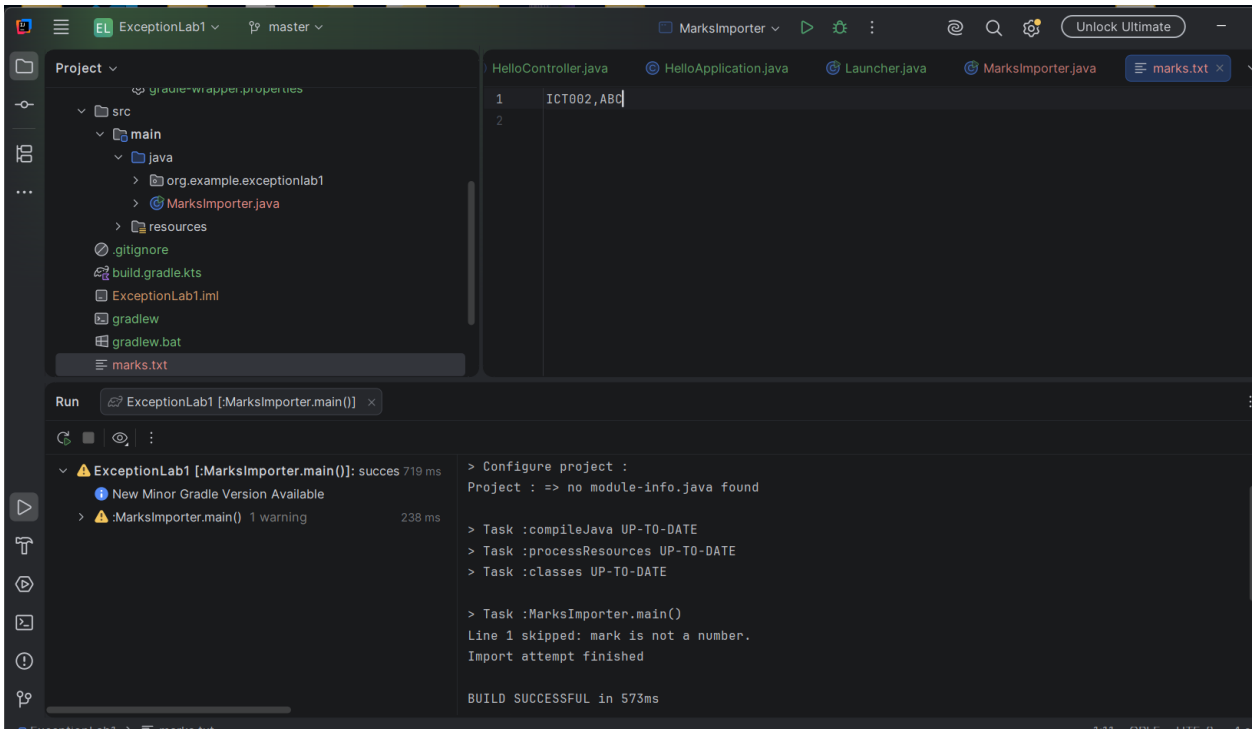
2. **T1**

3. 

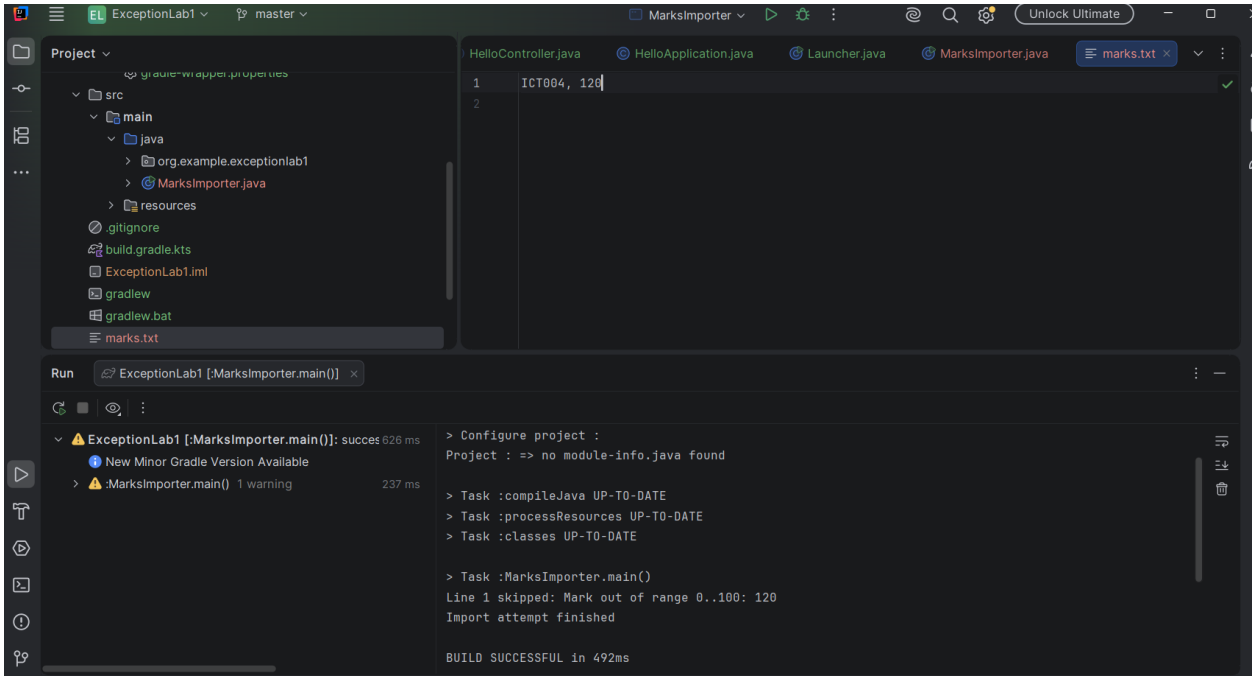
T2:


ExceptionLab1
master
MarksImporter
Unlock Ultimate
Project
HelloController.java
HelloApplication.java
Launcher.java
MarksImporter.java
marks.txt
src
main
java
org.example.exceptionlab1
MarksImporter.java
resources
.gitignore
build.gradle.kts
ExceptionLab1.iml
gradlew
gradlew.bat
marks.txt
import java.io.*;
import java.util.*;
import java.util.logging.*;
// Custom unchecked exception for repeated student IDs
class DuplicateStudentException extends RuntimeException { 3 usages
public DuplicateStudentException(String message) { 1 usage
super(message);
}
}
public class MarksImporter {
private static final Logger logger = Logger.getLogger(MarksImporter.class.getName()); 6 usage
Run
ExceptionLab1 [:MarksImporter.main()]
ExceptionLab1 [:MarksImporter.main()]: succes 554 ms
New Minor Gradle Version Available
> Configure project :
Project : => no module-info.java found
> Task :compileJava UP-TO-DATE
> Task :processResources UP-TO-DATE
> Task :classes UP-TO-DATE
> Task :MarksImporter.main()
Accepted: ICT001 -> 78
Import attempt finished
BUILD SUCCESSFUL in 415ms

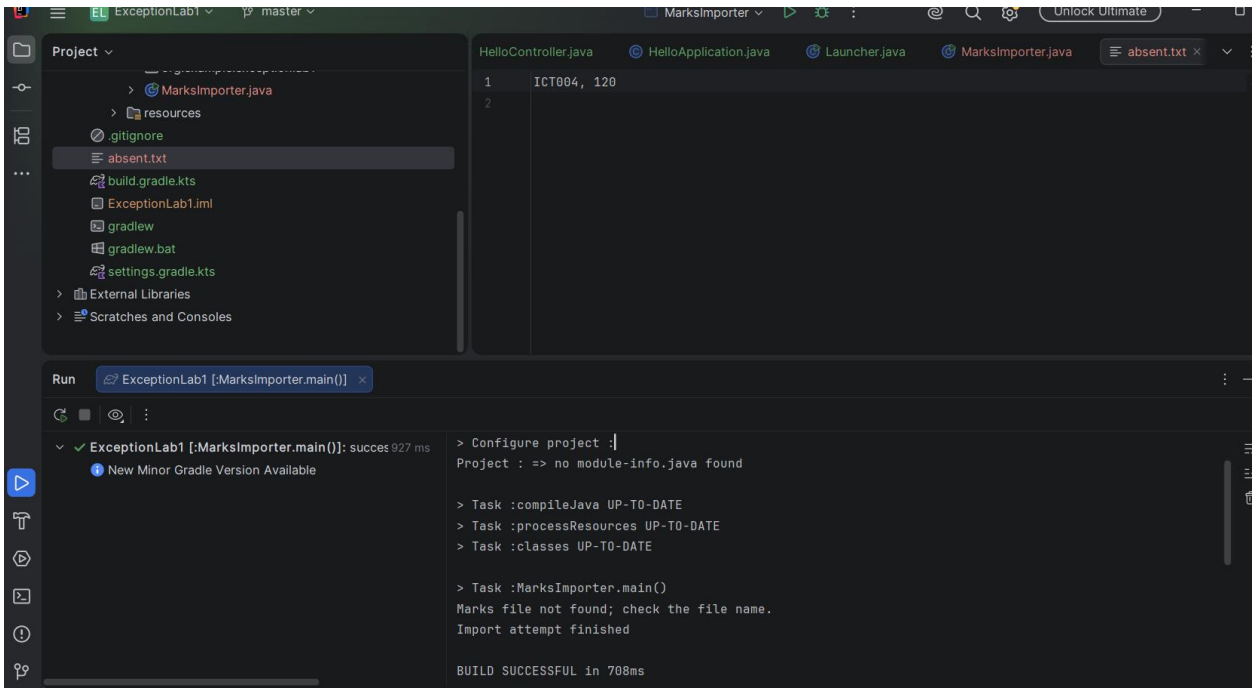
T3:


ExceptionLab1
master
MarksImporter
Unlock Ultimate
Project
HelloController.java
HelloApplication.java
Launcher.java
MarksImporter.java
marks.txt
ICT002,ABC
src
main
java
org.example.exceptionlab1
MarksImporter.java
resources
.gitignore
build.gradle.kts
ExceptionLab1.iml
gradlew
gradlew.bat
marks.txt
Run
ExceptionLab1 [:MarksImporter.main()]
ExceptionLab1 [:MarksImporter.main()]: succes 719 ms
New Minor Gradle Version Available
:MarksImporter.main() 1 warning 238 ms
> Configure project :
Project : => no module-info.java found
> Task :compileJava UP-TO-DATE
> Task :processResources UP-TO-DATE
> Task :classes UP-TO-DATE
> Task :MarksImporter.main()
Line 1 skipped: mark is not a number.
Import attempt finished
BUILD SUCCESSFUL in 573ms

T4:



T5:



ACTIVITY D:

1. Which of your exceptions is checked, and where does throws appear? Why?
FileNotFoundException and IOException are checked. throws IOException appears on the importMarks method signature. File reading can fail for reasons outside the program's control, so the compiler forces me to either handle the problem or declare it. My DuplicateStudentException is unchecked because it extends RuntimeException.

2. The difference between throw and throws
throw actually raises an exception at that moment:
throw new DuplicateStudentException("Duplicate student ID: " + id);
throws is a declaration in the method signature saying the method may pass an exception to its caller:
static void importMarks(String filename) throws IOException {

3.Why a friendly message for the user and technical details in the logger?
Users need to know what went wrong and what to do next, for example "check the file name". Raw technical details confuse them and can expose internal information about the program. Developers still need the exception type and cause to find and fix the bug, so the logger keeps those details in a record.

4. What goes wrong with an empty catch block, or with catching Exception before FileNotFoundException?
An empty catch block swallows the error silently. The program carries on in a bad state, nobody is told, and the problem becomes very hard to find. Catching Exception first makes the later FileNotFoundException catch unreachable, because Exception already covers it, so the compiler reports an error. Specific exceptions must come before general ones.